

一、什么是元数据

元数据（metadata）——关于数据的数据。它不是对数据主体内容的直接描述，而是对数据的属性、上下文、格式、来源等信息的系统性记录。简单来说，元数据帮助我们回答：这份数据是谁创建的？什么时候创建的？用什么格式存储的？应该如何使用它？

二、元数据架构与存储形式

一、存储架构

元数据的存储架构决定了它“住在哪里”，主要有三种模式：

- **嵌入式** — 元数据直接写在文件内部，如 JPEG 的 EXIF、PDF 的文档属性、MP3 的 ID3 标签。优点是自包含，文件到哪数据跟到哪；缺点是不易批量管理。
- **外部独立文件** — 元数据单独存储在旁边文件中，如 .xml、.json 伴随原文件。优点是易于编辑和批量处理；缺点是两个文件可能分离。
- **中央数据库 / 目录** — 元数据集中存储在数据库或目录系统中，通过唯一 ID 关联原文件。优点是支持强大的检索和批量管理；缺点是依赖数据库运行。

二、常见存储格式

元数据需要用结构化的方式表达，常见的存储格式包括：

格式	全称	特点	典型应用
JSON	JavaScript Object Notation	轻量、易读、广泛支持	Web API、NoSQL、配置文件
XML	eXtensible Markup Language	可扩展、Schema 验证	Dublin Core、METS、图书馆系统
RDF	Resource Description Framework	三元组模型、语义网基础	关联数据、知识图谱
YAML	YAML Ain't Markup Language	比 JSON 更可读	Dockerfile、CI/CD、文档头部

三、JSON 与元数据

JSON 本质上就是“键-值对”结构，这和元数据“属性=值”的本质完全一致。

```
{
  "title": "元数据介绍",
  "author": "张三",
  "date": "2026-08-10",
  "format": "docx",
  "language": "zh-CN",
  "keywords": ["元数据", "metadata"],
  "license": "CC BY 4.0"
}
```

四、常见元数据标准

为了让不同系统之间能互相理解元数据，业界制定了多种标准化方案：

- **Dublin Core** — 最基础的通用元数据标准，定义 15 个核心元素
- **METS** — 用于描述数字化对象的结构，常用于图书馆数字化项目
- **PREMIS** — 专注于数字保存，记录文件的生命周期、格式变迁、完整性校验等
- **Schema.org** — 由 Google、Bing、Yahoo 联合推出，用于网页中嵌入结构化数据，提升 SEO
- **DDI** — 专为社会科学数据设计，描述调查问卷、数据集、变量等

Dublin Core — 15 个核心元素

Dublin Core 由 DCMI 维护，所有元素均为可选、可重复：

元素名	中文名	说明	示例
dc:title	题名	资源的名称	metadata 介绍
dc:creator	创建者	创建资源的主要责任者	张三
dc:subject	主题	主题关键词或分类	元数据; Dublin Core
dc:description	描述	内容的摘要或说明	一份关于元数据的演示文档
dc:publisher	出版者	使资源可用的责任者	作者本人
dc:contributor	其他贡献者	次要贡献者	译者或审阅人
dc:date	日期	生命周期中的关键日期	2026-08-10
dc:type	类型	资源的性质或体裁	Text / 演示文档
dc:format	格式	文件格式或物理媒介	docx
dc:identifier	标识符	唯一标识	https://example.com/...

dc:source	来源	衍生出本资源的原始资源	基于某教材第三章改写
dc:language	语种	资源内容的语言	zh-CN
dc:relation	关联	与其他资源的关系	“数据管理入门”系列之一
dc:coverage	覆盖范围	空间或时间范围	中国; 2026 年
dc:rights	权限	知识产权和使用条款	CC BY 4.0

三、元数据的分类

根据功能和作用的不同，元数据通常分为三种类型：

一、描述性元数据

用于发现和识别资源，回答“它叫什么？谁做的？”

- **标题** — 资源的名称
- **作者 / 创建者** — 谁制作了这份资源
- **主题 / 关键词** — 资源涉及的主题标签
- **摘要 / 描述** — 内容的简要说明
- **语言** — 内容使用的语言
- **发布日期** — 资源发布或创建的时间
- **资源类型** — 如数据集、论文、图像、视频等
- **标识符** — 如 DOI、ISBN、URL 等唯一标识
- **覆盖范围** — 时间跨度或地理范围

二、结构性元数据

描述资源的内部组织结构，回答“它是如何组织起来的？”

- **目录 / 章节** — 文档的逻辑结构与层级关系
- **页码 / 顺序** — 内容排列的先后次序
- **卷 / 期号** — 连续出版物的组织方式
- **段落 / 节** — 文本的内部划分单位
- **关联资源** — 前一篇/后一篇、上一级/下一级的关系
- **嵌入资源** — 文档中嵌入的图片、表格、附件等

三、管理型元数据

用于资源的长期管理、保存和维护，回答“如何管理它？谁有权访问？”

- **文件格式** — 如 .docx、.pdf、.csv、.jpg
- **文件大小** — 存储空间占用信息
- **创建日期 / 修改日期** — 资源的时间戳
- **访问权限** — 谁可以查看、编辑或删除
- **版权 / 许可信息** — 使用条款和知识产权声明
- **技术环境要求** — 查看或运行该资源所需的软硬件
- **保存期限** — 资源的保留和销毁策略
- **数字签名 / 校验和** — 确保文件的完整性和真实性

四、元数据的核心功能

元数据的核心功能可以概括为六个字：发现、理解、管理。

- **发现与检索** — 让数据能被找到——搜索引擎、图书馆目录、数据集平台都靠它定位资源
- **识别与描述** — 告诉你“这是什么”——标题、作者、摘要帮助快速判断资源是否相关
- **组织与关联** — 理清数据之间的结构关系——目录、分卷、数据表关联都靠它
- **管理与保存** — 支撑长期维护——文件格式、创建时间、权限控制都属于这一类
- **互操作与交换** — 让不同系统能理解彼此的数据——标准化元数据让跨平台交换成为可能
- **可信度与溯源** — 明确数据来源和处理历史——谁创建的、经过什么处理、有没有被篡改

三种元数据对比总结

类型	核心问题	常见标签	应用场景
描述性元数据	它是什么？ 谁创建的？	标题、作者、关键词 摘要、标识符	搜索引擎、图书馆目录 数据集发现
结构性元数据	如何组织的？ 各部分什么关系？	目录、章节、页码 卷期号、关联资源	电子书导航、论文排版 数据库关系

管理型元数据	如何管理？ 谁能访问？	文件格式、创建日期 权限、版权信息	数字资产管理 合规审查、长期保存
--------	----------------	----------------------	---------------------

五、EXIF —— 照片中的元数据

每当你用手机或相机拍下一张照片，设备会自动在图片文件中嵌入大量元数据。这套标准叫做 EXIF（Exchangeable Image File Format），是最常见、最直观的元数据案例。

一、EXIF 记录了哪些信息

一张手机照片的 EXIF 通常包含以下信息：

分类	EXIF 标签	含义	示例值
描述性	Make / Model	设备品牌与型号	Apple iPhone 15 Pro
描述性	Software	处理软件	Adobe Lightroom
描述性	ImageDescription	图片描述	夕阳下的西湖
管理型	DateTimeOriginal	拍摄日期时间	2026-08-10 18:32:15
管理型	GPSLatitude / GPSLongitude	GPS 经纬度	30.2595 N, 120.1303 E
管理型	GPSAltitude	海拔高度	15.3 m
管理型	Copyright	版权信息	Zhang San
管理型	FileSize	文件大小	3.2 MB
结构性	ImageWidth / ImageHeight	图像分辨率	4032 x 3024
结构性	Orientation	旋转方向	Normal (1)
管理型	ISO / FNumber / ExposureTime	ISO / 光圈 / 快门	ISO 400, f/1.8, 1/120s
管理型	Flash	闪光灯状态	Off

二、EXIF 的隐私风险

EXIF 的强大之处也正是它的风险所在。一张原图分享出去，可能无意中泄露以下信息：

- **精确 GPS 坐标** — 暴露家庭住址、工作地点、常去场所
- **拍摄时间** — 推断作息规律、出行习惯
- **设备型号** — 暴露消费水平和设备偏好

- **软件信息** — 透露使用的编辑工具和版本

大多数社交平台（微信、微博、Instagram 等）上传时会自动剥离 EXIF，但通过网盘、邮件、即时通讯发送原图时，EXIF 信息通常会被完整保留。

三、EXIF 的操作与剥离

以下面的流程为例，讲解如何读取和剥离 EXIF 元数据。

① 读取 EXIF —— 使用 Pillow 库的 getexif() 方法：

```
图片对象 = 打开图片文件("photo.jpg")  
元数据字典 = 图片对象.获取 EXIF 信息()
```

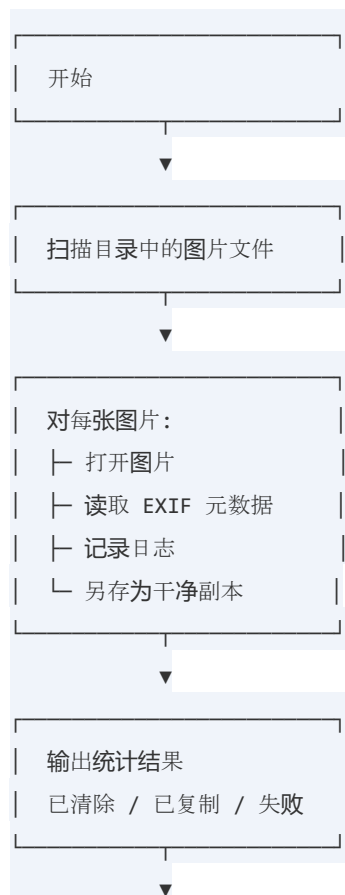
对于 每一个标签 ID，标签值 在 元数据字典中：

```
标签名称 = EXIF 标签名称表[标签 ID]  
输出(f"{标签名称}: {标签值}")
```

② 剥离 EXIF —— 保存时传入空的 exif 参数：

```
图片对象.保存("clean.jpg", 清除 EXIF 元数据=True)
```

③ 完整工作流程：



结束

核心逻辑：打开图片→读取 EXIF→记录日志→保存时写入空字节清除元数据。原始文件始终不受影响，干净版输出到单独目录。

六、标识符 —— 资源的“身份证”

从 HTML 的 id 说起

你一定用过 HTML 中的 id 属性：

```
<div id="section-3">这是第三章</div>
```

```
<a href="#section-3">跳转到第三章</a>
```

HTML 的 id 做了三件事：给元素一个唯一名字、让其他元素可以引用它、使得在整个页面中能精确定位它。这恰好就是元数据中“标识符”的核心思想。

实际场景：批量文件归档

开发中经常遇到这样的情况：一个批次产生了多个不同类型的文件，需要归档到同一个目录。比如某次实验的一组数据：

```
experiment_20260810/  
├ raw_data.csv      # 原始数据  
├ config.json       # 配置参数  
├ result.png        # 结果图片  
├ analysis.ipynb    # 分析笔记本  
└ metadata.json     # 元数据
```

问题是：如果需要批量处理多个批次的文件，程序怎么知道哪些文件属于同一组？

靠文件名？不靠谱。文件名可能被修改、重命名、缺失、编码乱码。正确的做法是：每个文件携带一个标识符，程序读标识符而不是读文件名。

自定义标识符设计

既然没有现成的标准，就自己定义一套。设计原则很简单：

- **可读性** — 人能看懂，方便调试和日志排查
- **唯一性** — 同一批次内不重复，跨批次也不重复
- **可解析** — 包含必要信息（项目、日期、序号），方便程序拆分
- **跨文件通用** — 同一个标识符可以嵌入到不同类型的文件中

一个典型的自定义标识符格式：

```
# 格式：{PROJECT}_{DATE}_{BATCH}_{SEQ}
EXP_20260810_A001_0001
EXP_20260810_A001_0002
EXP_20260810_A002_0001
```

```
# 拆分解读：
#   EXP      - 项目代码
#   20260810 - 日期
#   A001     - 批次号
#   0001     - 批次内序号
```

在不同文件中嵌入标识符

同一个标识符可以写入不同类型的文件，方式各不相同：

文件类型	嵌入方式	示例
JSON / YAML	直接写入字段	"batch_id": "EXP_20260810_A001"
CSV	新增一列	batch_id 列，每行填入标识符
图片	写入 EXIF 或文件名前缀	EXP_20260810_A001_0001.png
任意文件	伴随 .meta 文件	同名 .meta 记录标识符

程序中的匹配逻辑

程序启动时，扫描目录中所有文件，提取标识符，按标识符归组：

```
函数 加载批次文件(目录路径):
    批次字典 = 新建空字典

    对于 目录中每一个文件:
        批次 ID = 提取标识符(该文件)
        如果 批次 ID 不为空:
            将文件添加到 批次字典[批次 ID] 的列表中

    返回 批次字典
```

执行结果示例：

```
"EXP_20260810_A001" → [raw.csv, config.json, result.png]
"EXP_20260810_A002" → [raw.csv, config.json, result.png]
```

提取标识符的实现逻辑：

函数 提取标识符(文件路径)：

```
扩展名 = 文件路径.后缀名.转小写()
```

情形 扩展名：

等于 ".json"：

```
数据 = 读取 JSON(文件路径)
返回 数据.字段["batch_id"]
```

等于 ".csv"：

```
表格 = 读取 CSV(文件路径, 仅首行)
如果 表格.包含列("batch_id")：
    返回 表格["batch_id"].首行值
```

属于 (".png", ".jpg")：

```
图片 = 打开图片(文件路径)
EXIF 数据 = 图片.获取 EXIF()
返回 EXIF 数据[用户注释字段] // 0x9286
```

伴随文件 = 文件路径.追加后缀(".meta")

如果 伴随文件.存在()：

```
返回 读取文本(伴随文件).去除空白()
```

返回 空 // 未找到标识符

这样，程序就不再依赖文件名，而是通过标识符来关联同一批次的所有文件。文件名随意改，只要内部的标识符不变，匹配就不会出错。